Assignment 2A

Problem Statement

Design and develop a network topology of a 100 Mbps Fast Ethernet LAN having two 24-Port Switches. Each of the Switches is connected with a disjoint set of 10 nodes/ PCs configured with a class C IP Address (192.167.4.41-60). Also add PDU to check ping implementing simulation using Cisco Packet Tracer.

Theory

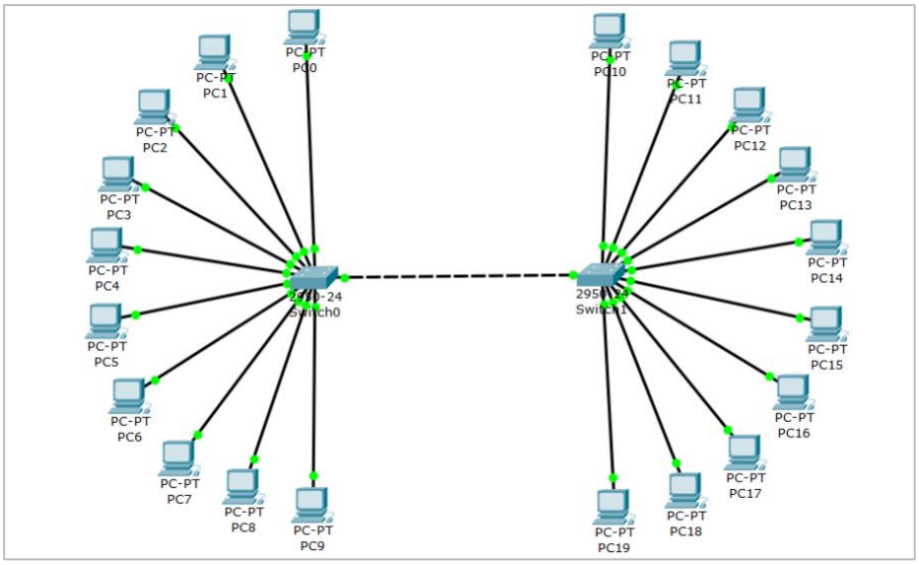
Cisco Packet Tracer is a network simulation tool developed by Cisco for learning and practicing networking concepts. It allows users to design, configure, and troubleshoot network topologies in a virtual environment. Packet Tracer supports various networking devices such as routers, switches, PCs, and IoT devices, enabling users to simulate real-world networking scenarios. It is widely used in networking courses, such as Cisco's CCNA, to enhance practical understanding without requiring physical hardware.

Benefits of Packet Tracer:

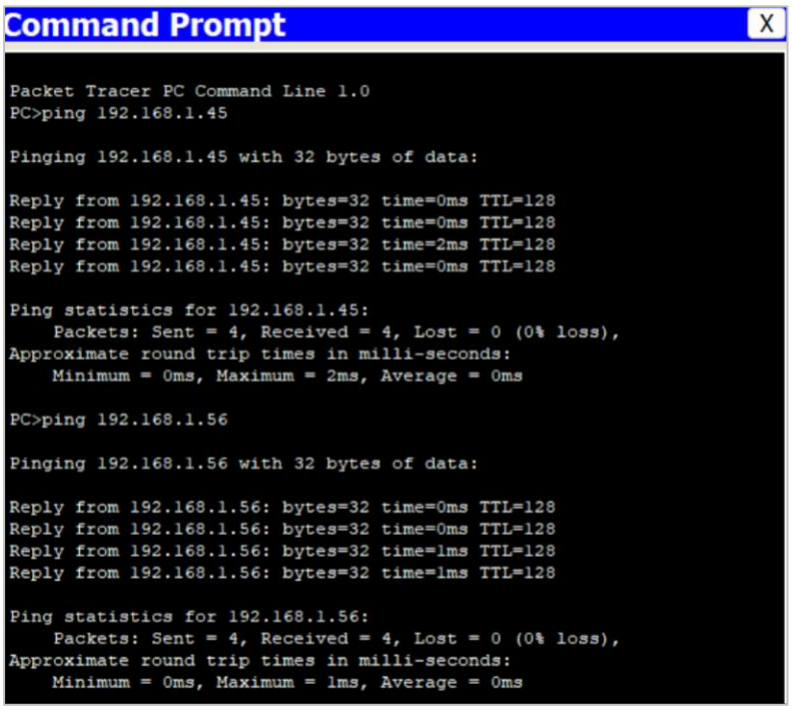
- Provides hands-on networking practice in a virtual environment.
- Eliminates the need for physical hardware, reducing costs.
- Supports network simulation, troubleshooting, and visualization.
- Available on multiple platforms for flexible learning.
- Includes IoT simulation and Python programming.
- Helps in CCNA preparation and real-world networking.
- Enables collaboration and remote learning.
- Scalable for small and large networks.

Procedure

1. After opening ***Cisco Packet Tracer***, we selected two switches which have 24 ports each and connected them with each other with a cross over copper cable.
2. Then we have taken 10 PCs for each switch and connected with their respective switch by straight through copper cable.

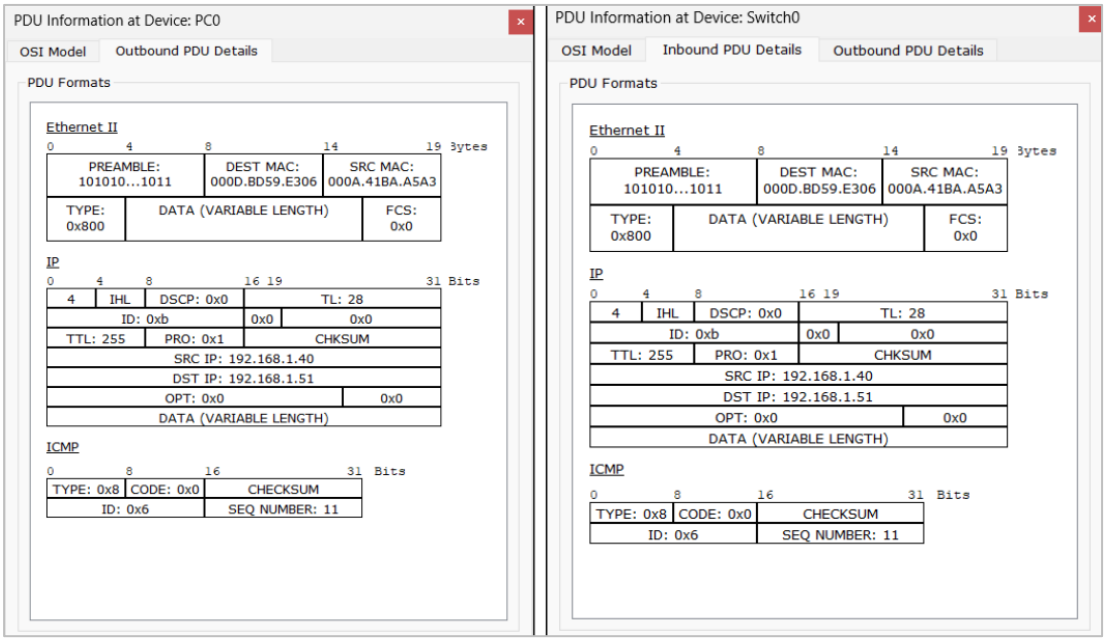


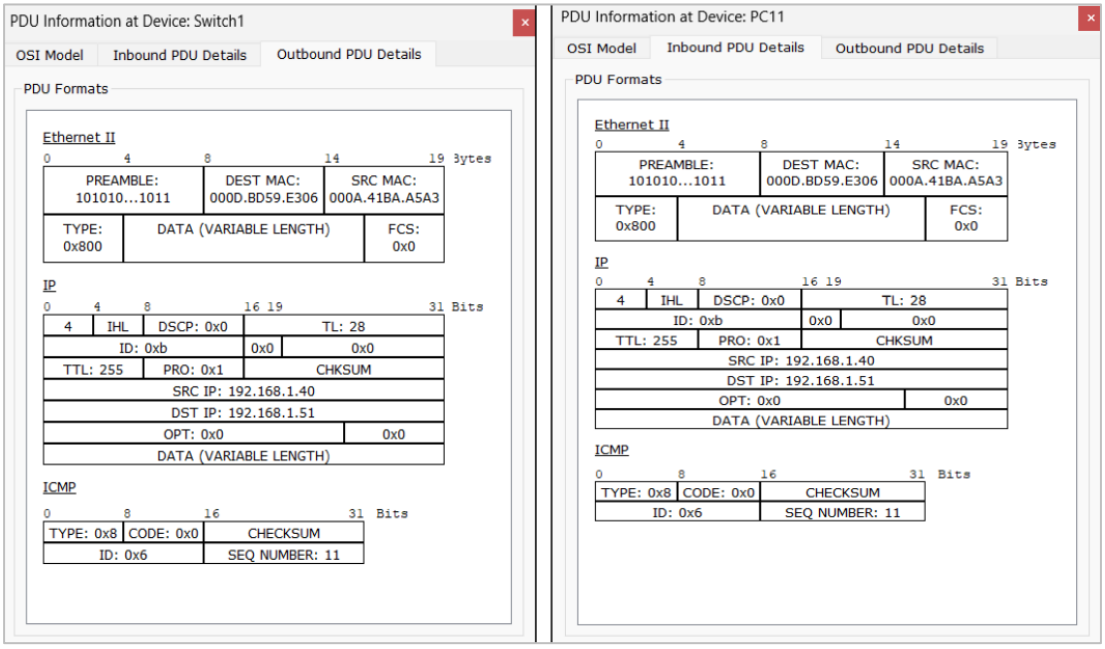
3. For configuration, we configured each PC's IP address with the IP address between 192.168.4.41 to 192.168.4.60. Then we used the simulated terminal to ping other devices in the network.



4. Then, we proceed with the simulation. During the simulation process, we check the network using different types of protocols. Many of these protocols are accepted by our constructed network, confirming its correctness.

5. We then sent a PDU packet from a PC from one switch to a PC from another switch which help us to check the connection of network.





Assignment 2B

Problem Statement

Configure your PC for LAN in Windows and Linux environment (configuration for Ethernet card).

Theory

An **IP (Internet Protocol) address** is a unique numerical identifier assigned to a device on a network, allowing it to communicate over the internet or a local network. It can be IPv4 (e.g., 192.168.1.1) or IPv6 (e.g., 2001:db8::1).

A **MAC (Media Access Control) address** is a hardware address assigned to a device's network interface card (NIC) by the manufacturer. It is a 48-bit address written in hexadecimal format (e.g., 00:1A:2B:3C:4D:5E).

While the IP address can change based on network configurations, the MAC address is unique to a device and remains constant. IP addresses operate at the Network Layer, whereas MAC addresses function at the Data Link Layer of the OSI model.

Procedure

A. Windows System

1. Go to ***Settings***.
2. Navigate to **Ethernet**.
3. Click on ***Change Adapter Settings***.
4. Right-click on the network adapter and select ***Properties***.
5. Click on ***Internet Protocol Version 4 (TCP/IPv4)***.
6. Click on ***Properties***.
7. Select **"*Use the following IP address*"** (i.e., static IP).
8. Set the ***IP address*** to 192.168.3.51 (Here, 192.168.3 is the Network ID and 51 is the Host ID – user-defined as required).
9. After entering the IP address, the ***Subnet Mask*** is automatically set to 255.255.255.0.
10. Set the ***Default Gateway*** as 192.168.3.1.

Since we are using a Class C type IP address, if we connect Ethernet to a single machine, the default gateway is optional. However, for a network setup, it is necessary.

B. Linux System

1. Open a terminal and type **"nmtui"** to launch the Network Manager Text User Interface:
2. Use the arrow keys to navigate and select **Edit a connection**, then press **Enter**.
3. Choose the network interface **eth0**, then select **Edit** and press **Enter**.
4. Scroll to the **IPv4 Configuration** section and set it to **Manual**.
5. Configure the following settings:
6. **IP Address**: 192.168.3.222
7. **Subnet Mask**: 255.255.255.0
8. **Default Gateway**: 192.168.3.1
9. Scroll down to the **DNS Servers** field and set:
10. **Primary DNS**: 8.8.8.8
11. **Secondary DNS**: 4.2.2.2
12. Select **OK**, then press **Enter**.
13. Back in the main menu, navigate to **Activate a connection**, select **eth0**, and deactivate it.
14. Reactivate **eth0** to apply the changes.
15. Exit nmtui.
16. Your network settings should now be updated.

Assignment 2C

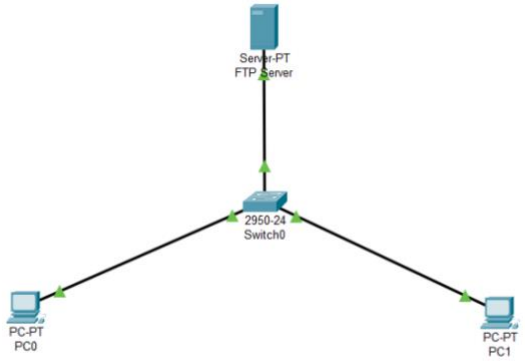
Problem Statement

Configuration of Router and FTP, HTTP, DNS and Mail Server

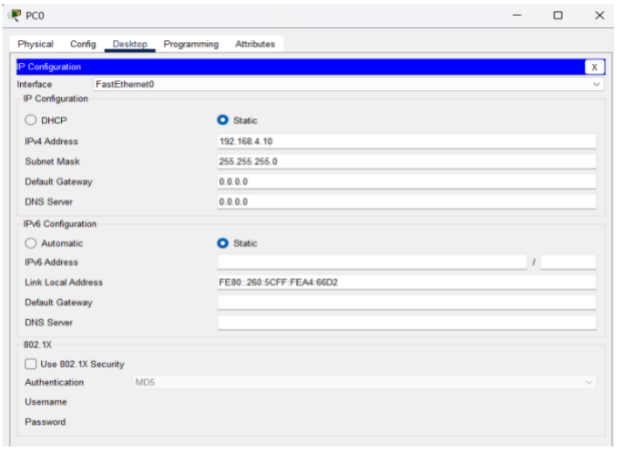
Procedure

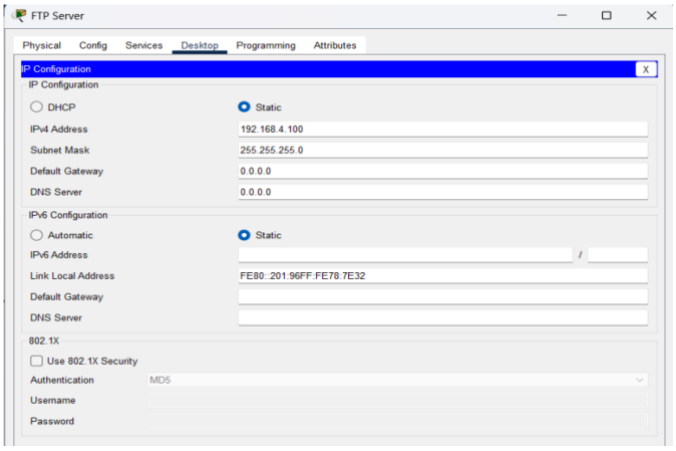
A. Configuration of FTP (File Transfer Protocol) Server:

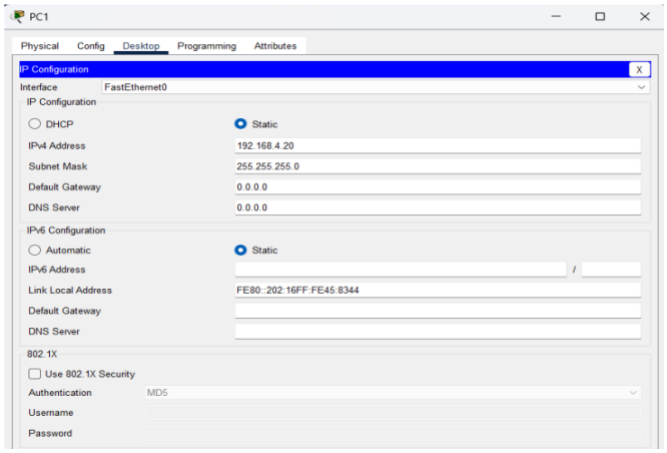
1. We make a Star Topology network of 3 nodes consisting of one Server to host the FTP Service and two PCs. We interconnect them with a 24 pin Switch



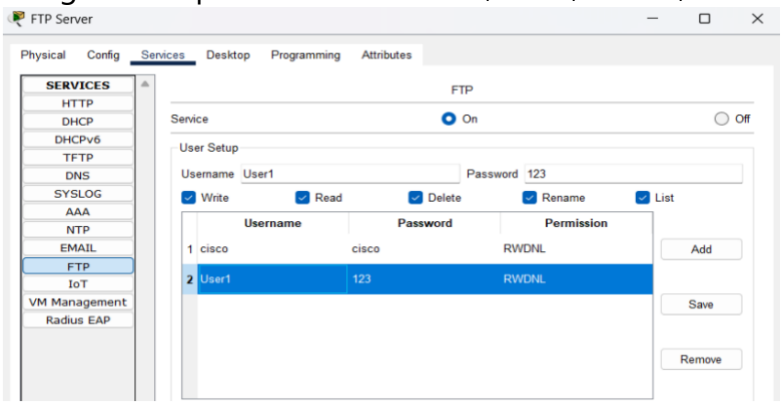
2. We assign IP Addresses to the three nodes of the network such that all of them belong to the same Network (having same network id, 192.168.4)



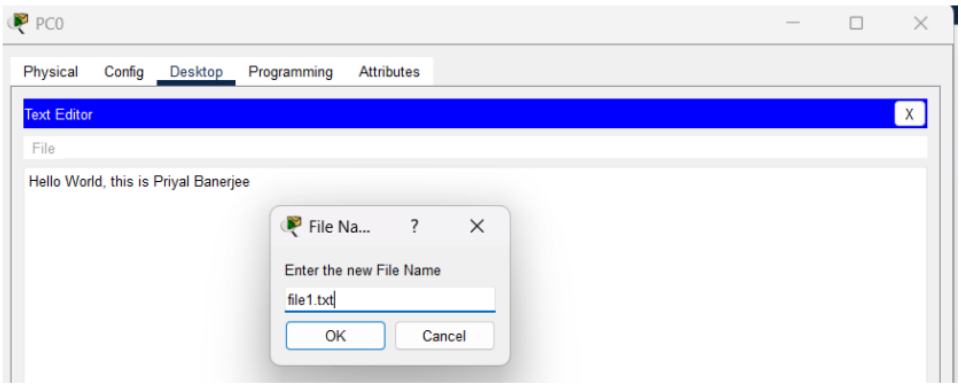




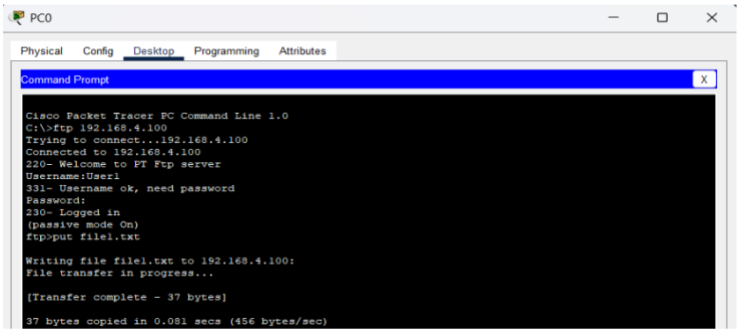
3. We assign the IP Addresses 192.168.4.100, 192.168.4.10 and 192.168.4.20 to the Server, PC0 and PC1 respectively
4. Then we configure the Server to enable the FTP Services from the Server's Services Menu
5. We create a new user in the Server with Username as User1 and Password as 123, and give it all permissions to Write, Read, Delete, Rename and List (RWDNL) for FTP



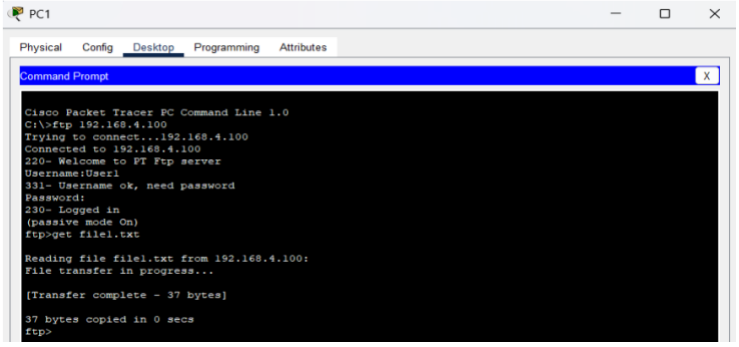
6. Then to test the FTP, we create a simple Text File in PC0 and save it as file1.txt



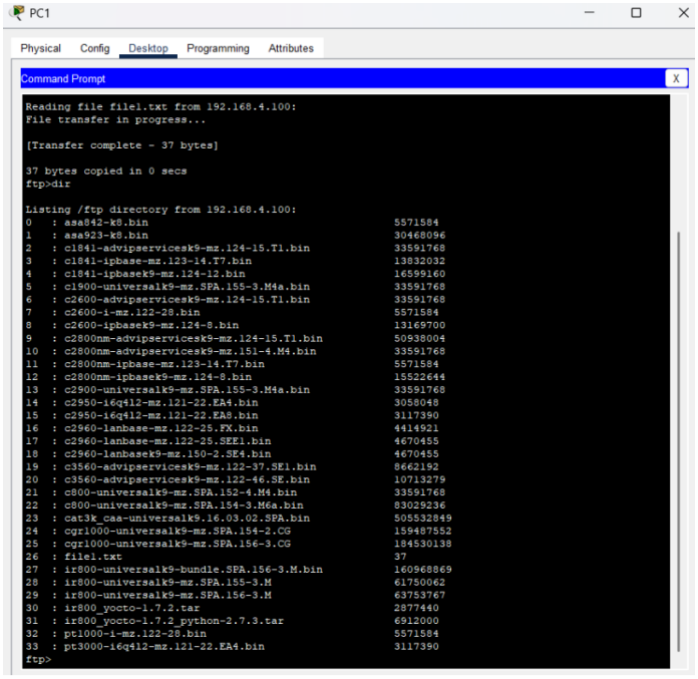
7. Then we connect to the Server using FTP Protocol from PC0 (using the command “ftp 192.168.4.100”) using the username and password set previously. Then upload file1.txt from PC0 to the Server using the command “put file1.txt”



8. Then we connect to the FTP Server from PC1, in the same way as we did from PC0 and download the file file1.txt in PC1 from the Server. We use the command “get file1.txt” to download the file

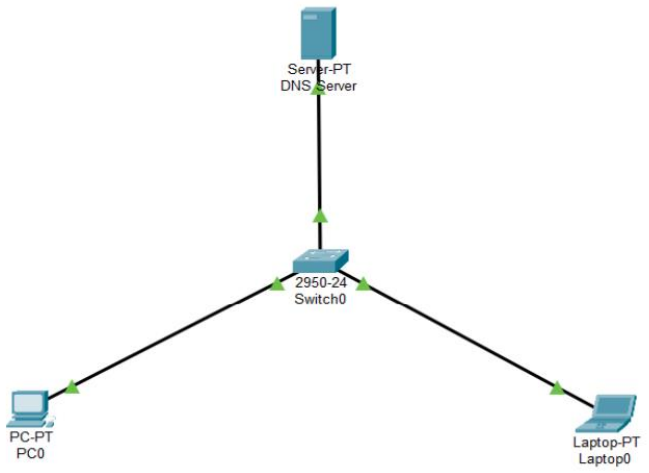


9. And we can then see the file is downloaded into PC1. This is how we can transfer a file from one PC to another using an FTP Server

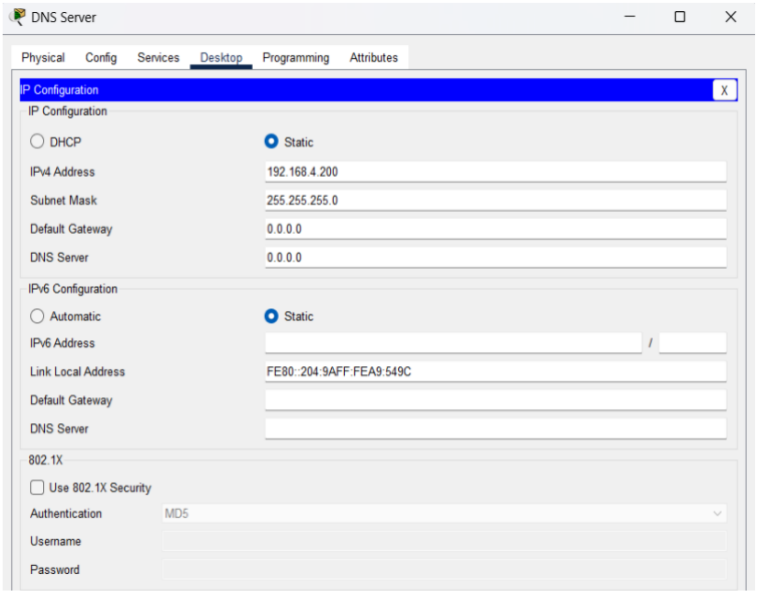


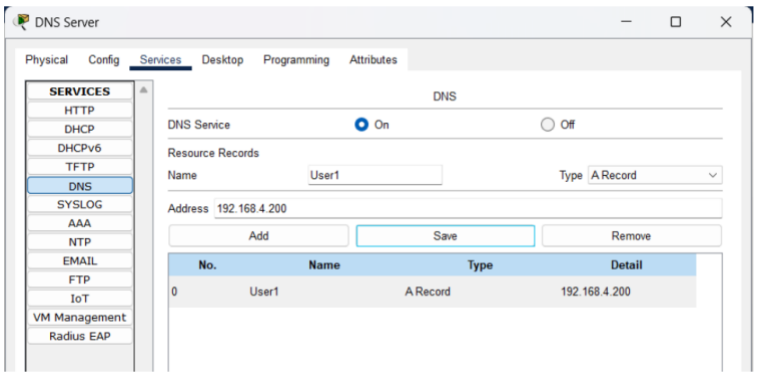
B. Configuration of DNS (Domain Name System) Server

1. We use the same Star Topology Network of one Server an two Workstations (one PC and one Laptop) interconnected with a 24 pin Switch for this

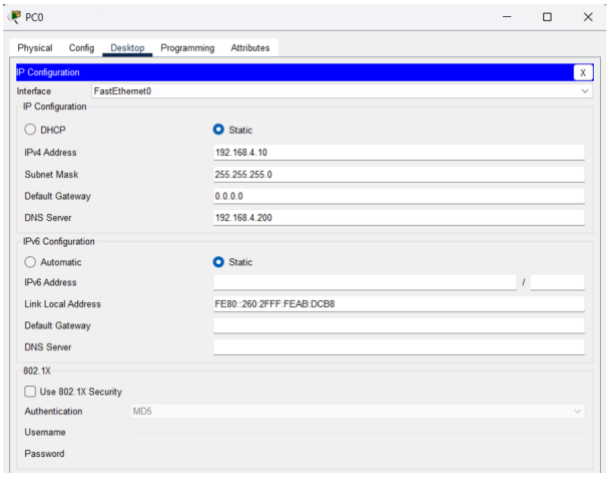


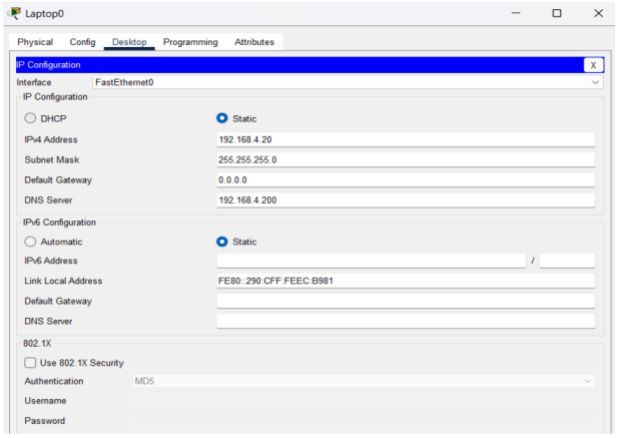
2. Then we assign IP Address to the Server (192.168.4.200) and enable its DNS Service so that the Server can work as a DNS Server. We set a Domain Name called "User1" with the IP of the Server



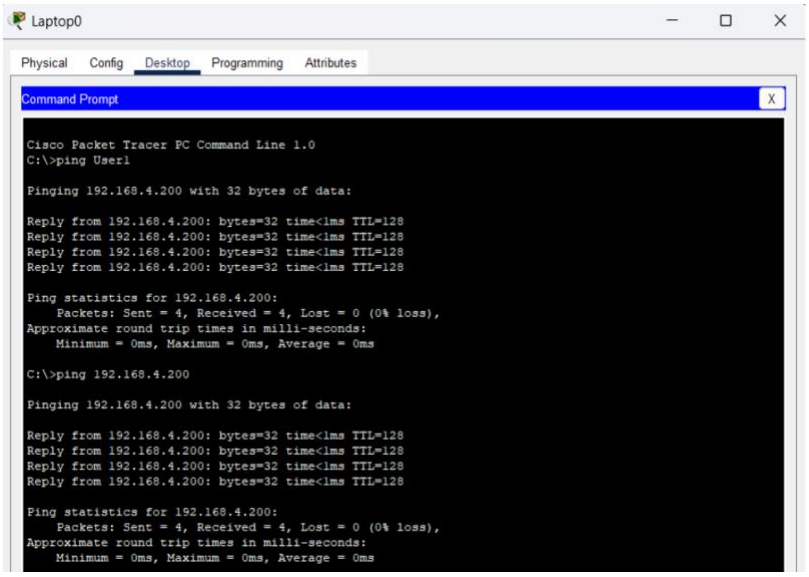


3. Then we assign IP Addresses to the Workstations, ensuring that they are in the same network as that of the Server
   PC0 → 192.168.4.10
   Laptop0 → 192.168.4.20
   We put the Server's IP in the DNS Server field of both the workstations, so that they can connect to that DNS



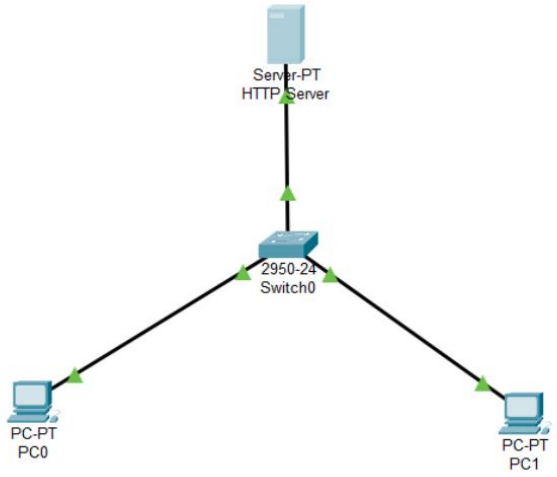


4. Then we check the DNS Connectivity using the "ping" command from Laptop0

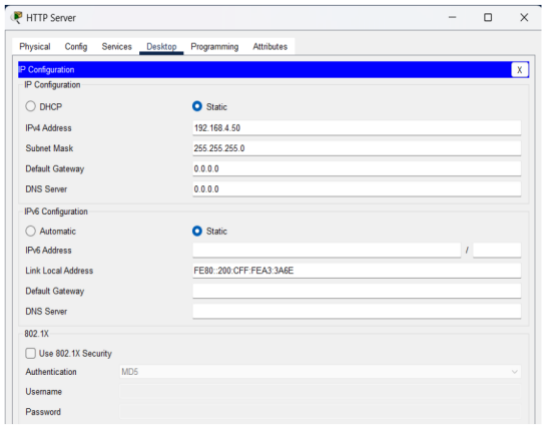


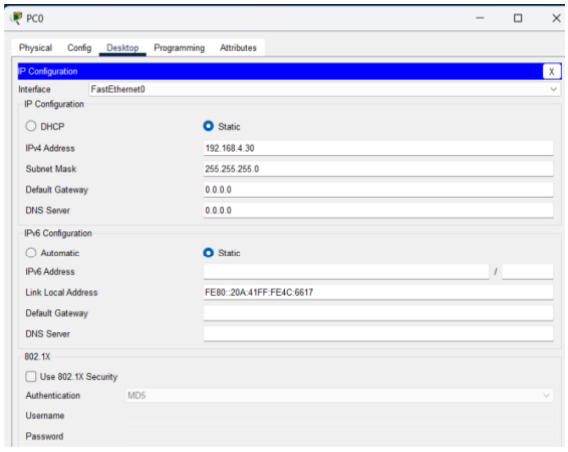
C. Configuration of HTTP (Hyper Text Transfer Protocol) Server

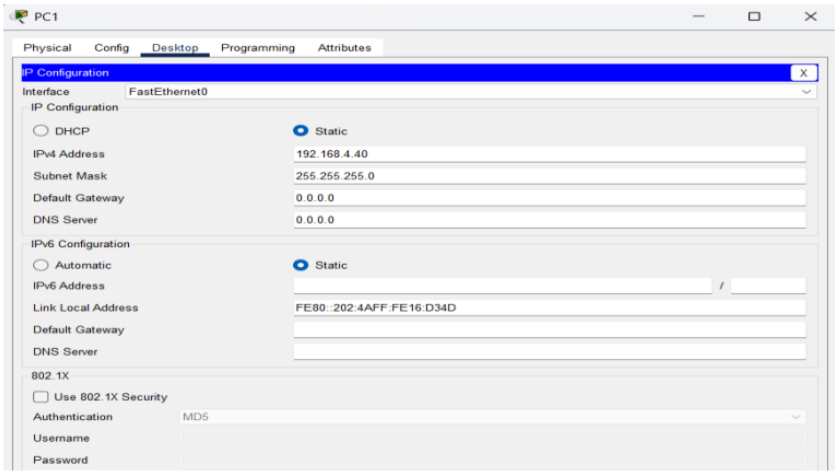
1. Configuring the same Star Topology Network for HTTP Services



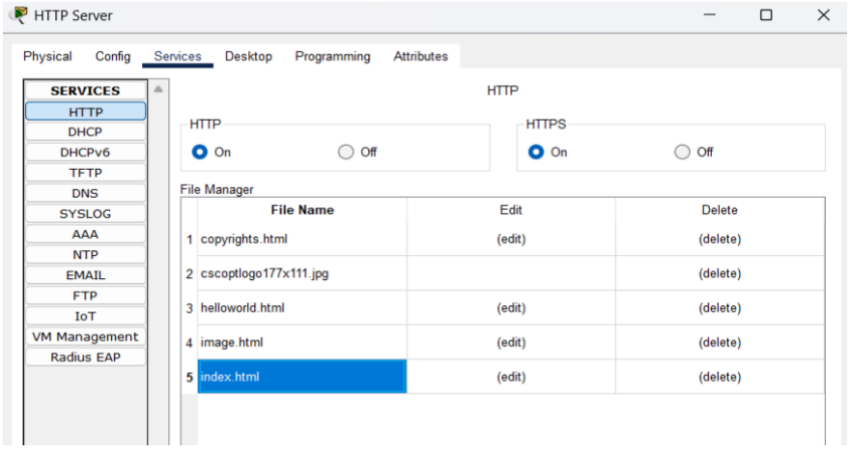
2. Changing the IP Addresses of the Server, PC0 and PC1 within the same network



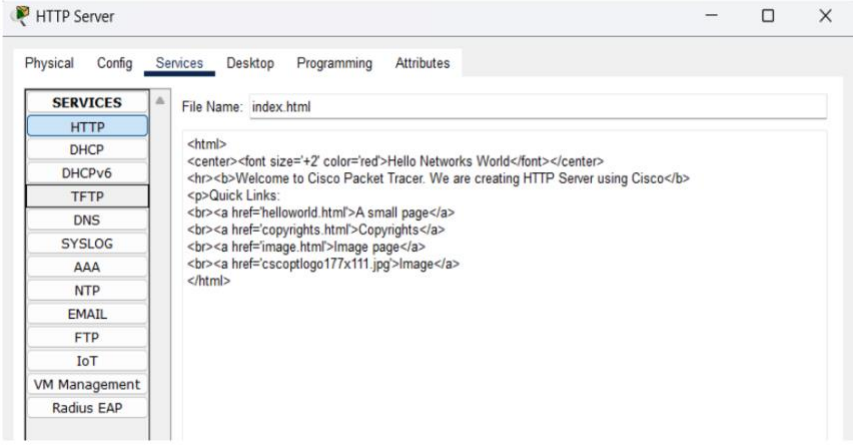




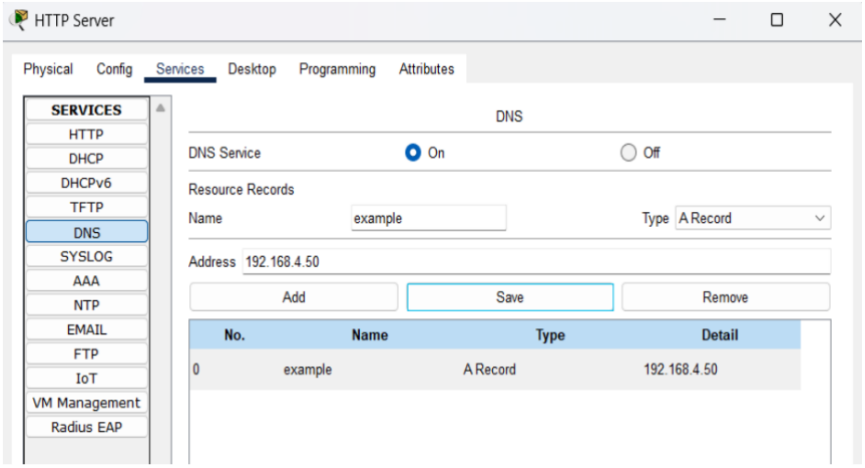
3. Enabling the HTTP Service from the Server and configuring the index.html page of the Server



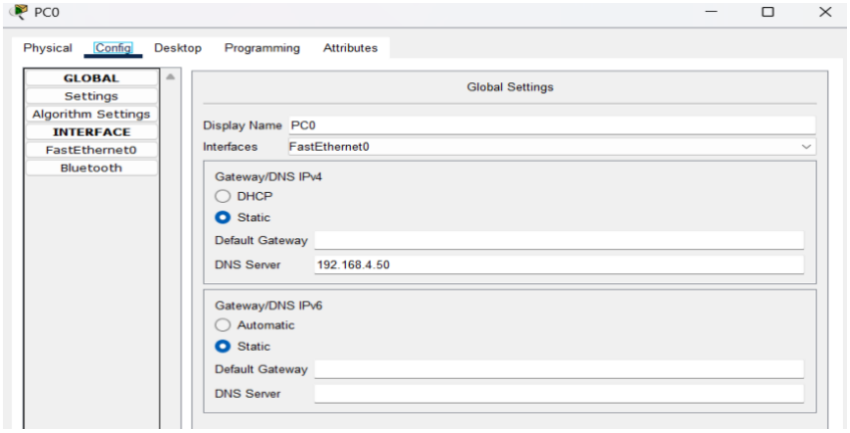
4. Configuring the index.html page to change the visuals, like bolding the texts, etc.



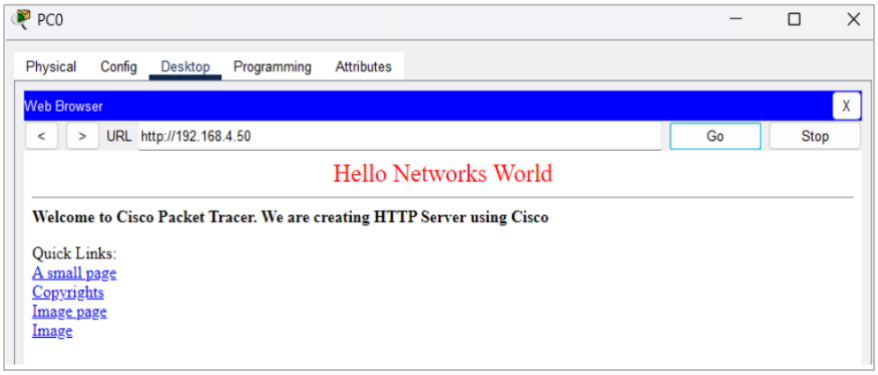
5. Setting up a DNS Server in the same Server, so that the HTTP Service of the Server can be tested using the Web Browser of another device by writing the site name only, and not explicitly mentioning the Server IP Address

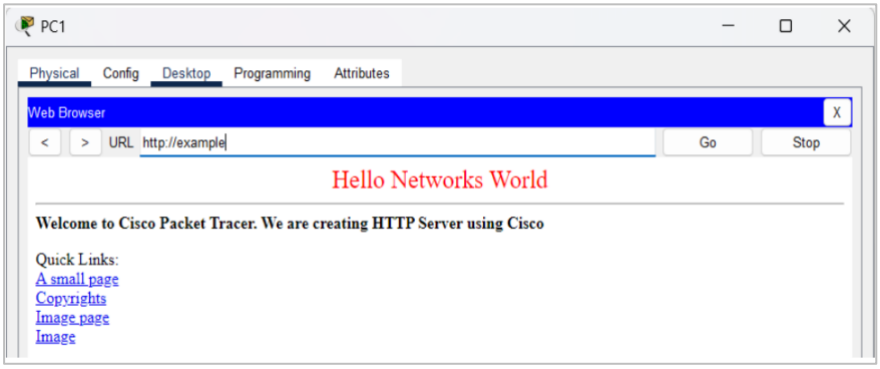


6. Whenever the site name “example” would be written on the web browser, it will automatically get replaced by the Server IP
7. Now, considering PC0 as the test PC, we connect it to the DNS Server to access the DNS Services



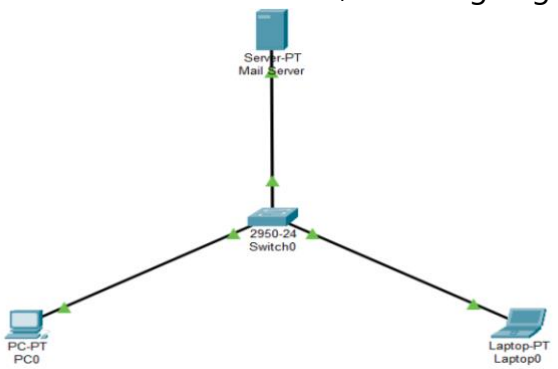
8. Now we open the Web Browser of both PC0 and PC1, and try connecting to the index.html page of the Server using the Server’s HTTP Services. We use the DNS name “example” in PC0 and direct IP Address 192.168.4.50 in the Web Browser’s Search Engines and we observe the same result for both

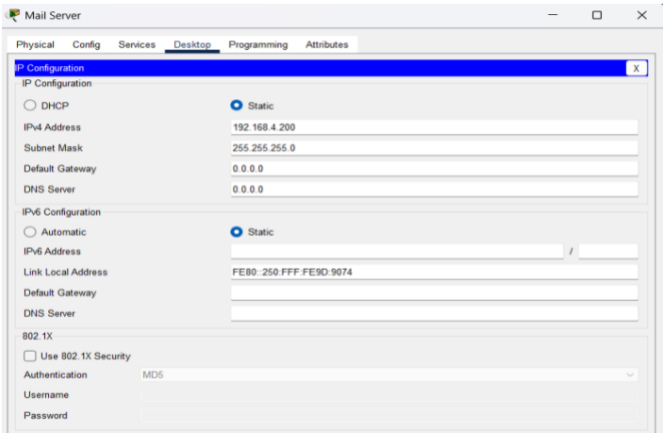


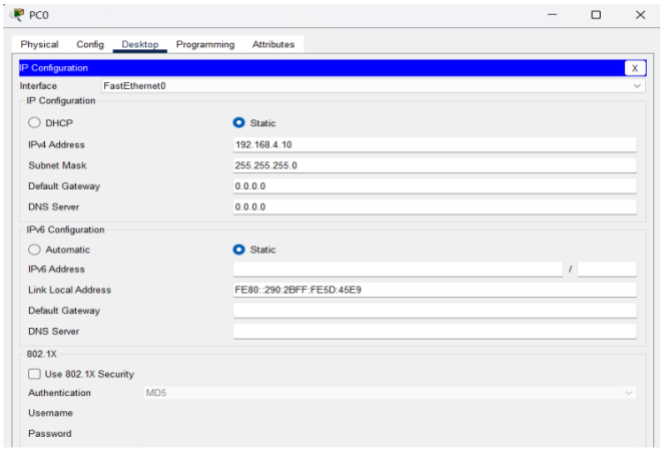


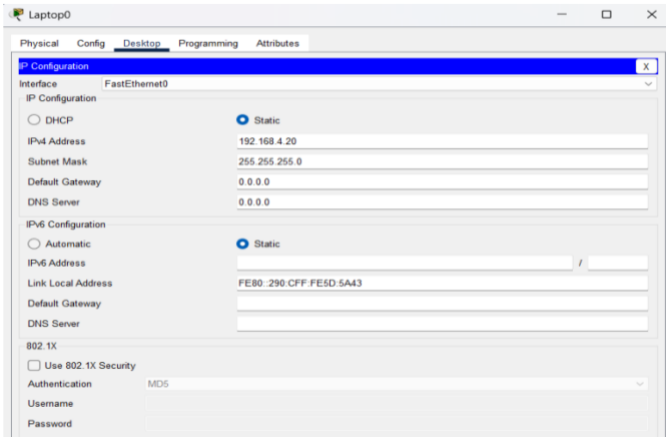
D. Configuration of Mail Server

1. Using the same Start Network, and assigning the IP Addresses to the workstations

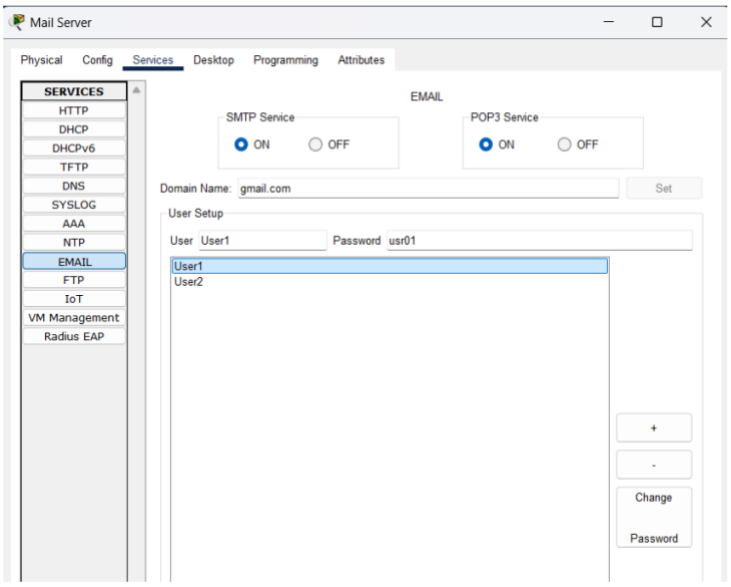




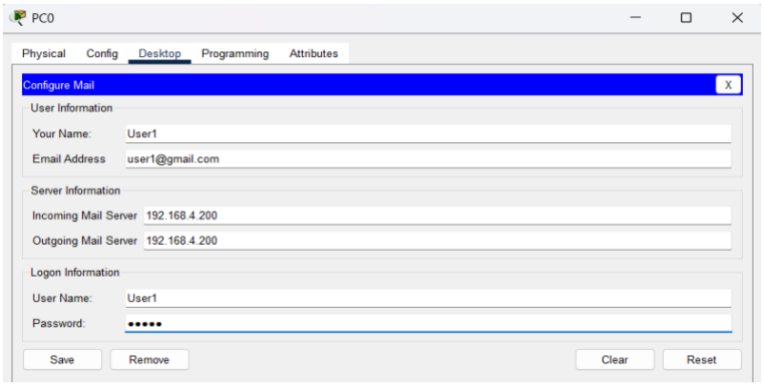


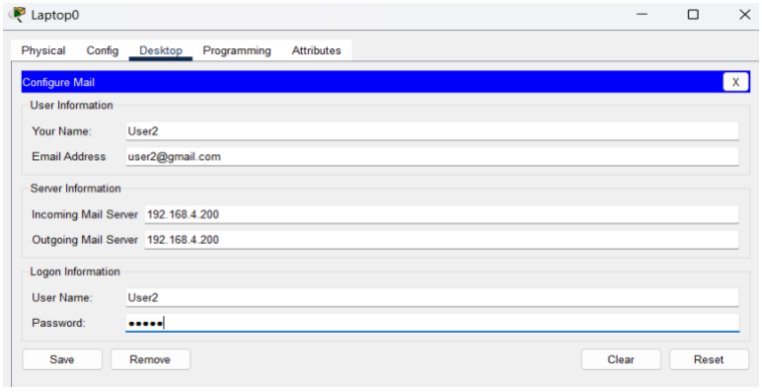


2. Mail Server IP Address = 192.168.4.200
   PC0 IP Address = 192.168.4.10
   Laptop0 = 192.168.4.20
3. Then we configure the Mail Server by turning on its Mail Services. And we create two users “User1” and “User2” in the Server

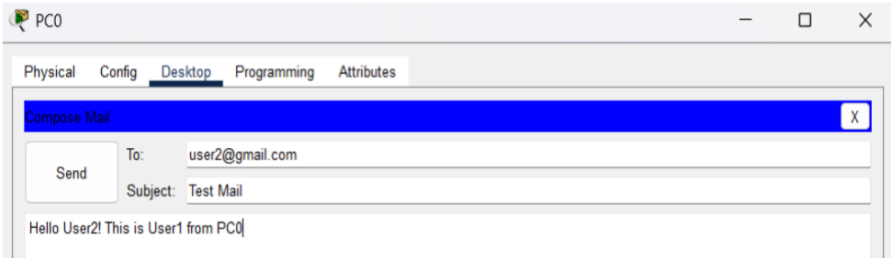


4. Then we configure the Mail Settings from both PC0 and Laptop0, such that both the Machines can work as the “User1” and “User2” of the Mail Server respectively. Assigning the email addresses of the machines (users) and passwords along with setting the Mail Server IP

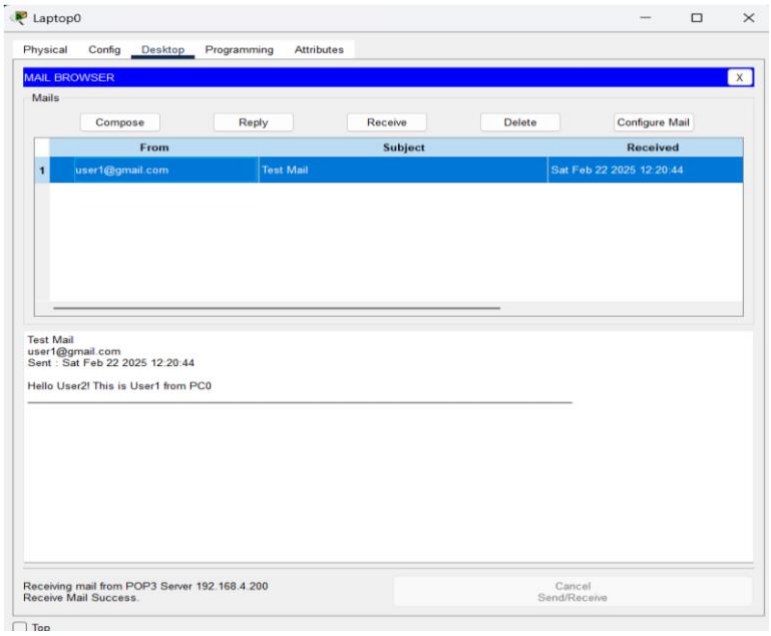




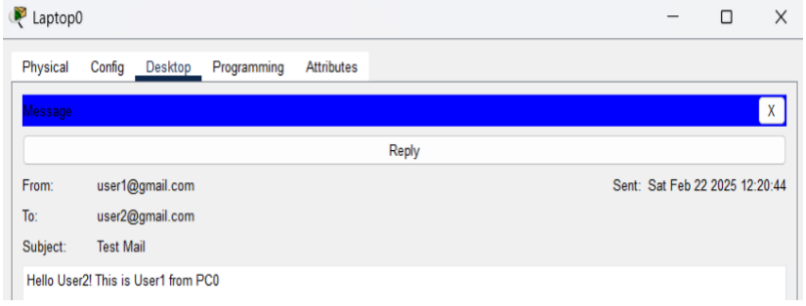
5. Now to test the Mail Server connectivity, we compose an email in PC0 and send it to the email address of Laptop0, via the Mail Server



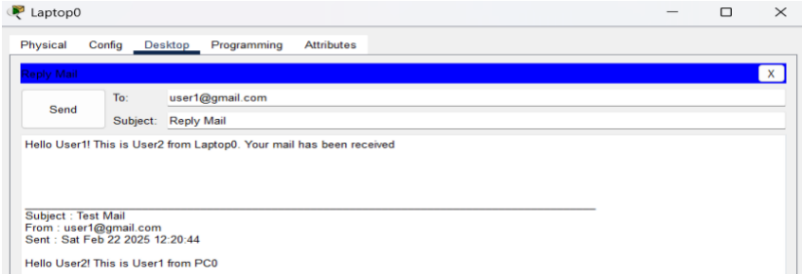
6. The mail has been successfully received by Laptop0 from PC0



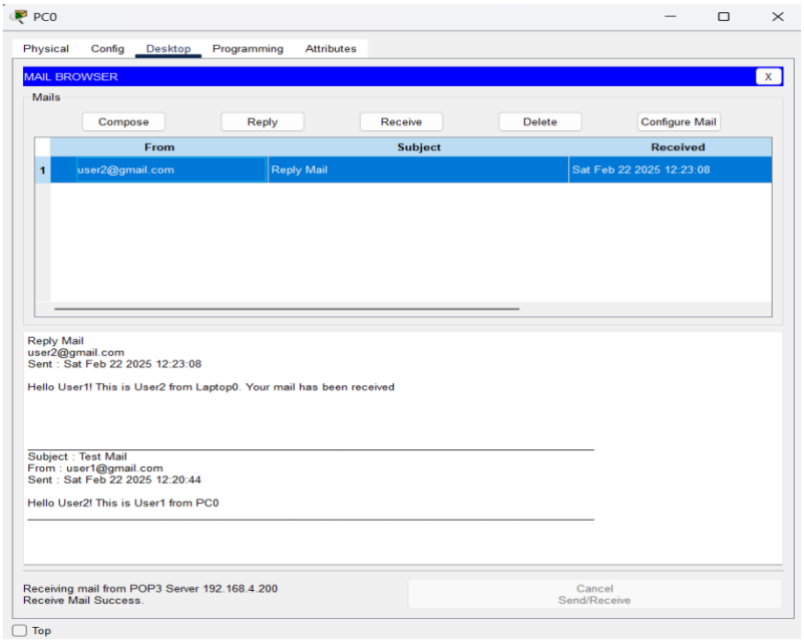
7. Checking the Mail content from Laptop0



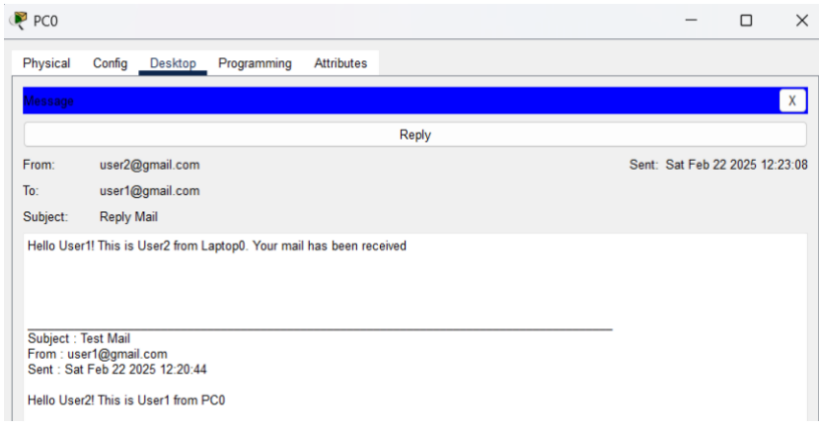
8. Now composing a reply mail for PC0 from Laptop0 and sending it



9. The reply mail from Laptop0 has been successfully received by PC0



10. Then we access the Mail from PC0



Assignment 3

Problem Statement

Write a program using pipe such that the content of an existing file to be written to the one end of the pipe and to be read from the other end of it.

Theory

IPC:

IPC is a technique to communicate between processes by the OS at rudimentary level. The concept has following applications.

I. Sending messages as well as signals between two processes.

II. Communication between Operating System commands.

PIPE

PIPE is an example of Interprocess communication or IPC. Pipe() is a system call and it is used for communication between two processes in half-duplex mode. It is also considered as file as both read and write operations can be performed on it. The syntax to create a pipe is as follows:

```
int pipe (int field[2]);
```

pipe takes an array of 2 integers as its only arguments. On successful execution it assigns two files descriptors. Whatever written in field[1] can be read in field[0], vice versa.

System call

The tool used for low-level programming is system call operating system's all its services using special type of functions, which are inbuilt to the kernel and perform basic functions. These require communication with hardware and software & resources available. Users can write program by allowing system calls in UNIX on UNIX like platform using assembly level language or C-like interface.

Code

```
#include <stdio.h>
#include <fcntl.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>

int main() {
    char s1[512], s2[512];
    int fd1, fd2[2], n, p, fd;
    fd1 = open(“a1.txt”, O_RDONLY);
    n = read(fd1, s1, 512);
    pipe(fd2);
    p = fork();
    if (p == -1) {
        printf("Fork Unsuccessful");
        exit(1);
    } else if (p == 0) {
        printf("In Child Now...\n");
        close(fd2[1]);
        n = read(fd2[0], s2, 512);
        printf("Received Data = %s\n", s2);
        fd = open("S1.txt", O_WRONLY | O_CREAT, 0644);
        write(fd, s2, n);
    }
    else {
        printf("In Parent Now...\n");
        close(fd2[0]);
        write(fd2[1], s1, n);
    }
}
```

Output

```
sujal@DESKTOP-BAENFT3:~$ vi pipe3.c
sujal@DESKTOP-BAENFT3:~$ gcc -o pipe3 pipe3.c
sujal@DESKTOP-BAENFT3:~$ cat > a1.txt
Hello World!
sujal@DESKTOP-BAENFT3:~$ ./pipe3
In Parent Now...
In Child Now...
Received Data = Hello World!
```

Assignment 4

Problem Statement

Write a program using TCP socket such that a client will accept a message from the user and send it to the server. The server will display the message and the IP address and port number of the client.

Code

In client.c:

```
#include <stdio.h>
#include <string.h>
#include <sys/socket.h>
#include <sys/types.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <unistd.h>
#define CLIENT_IP "127.0.0.1"
#define CLIENT_PORT 5671
#define SERVER_IP "127.0.0.1"
#define SERVER_PORT 5670
void main()
{
    struct sockaddr_in client, server;
    int sd;
    char msg[512];
    bzero((char *)&server, sizeof(server));
```

```
    server.sin_family = AF_INET;
    server.sin_port = htons(SERVER_PORT);
    server.sin_addr.s_addr = inet_addr(SERVER_IP);
    bzero((char *)&client, sizeof(client));
    client.sin_family = AF_INET;
    client.sin_port = htons(CLIENT_PORT);
    client.sin_addr.s_addr = inet_addr(CLIENT_IP);
    sd = socket(AF_INET, SOCK_STREAM, 0);
    connect(sd, (struct sockaddr *)&server, sizeof(server));
    do
    {
        printf("Enter a Message: ");
        scanf("%s", msg);
        send(sd, msg, strlen(msg) + 1, 0);
    } while (strcmp(msg, "stop"));
    close(sd);
}
```

In server.c:

```
#include <stdio.h>
#include <string.h>
#include <sys/socket.h>
#include <sys/types.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <unistd.h>
#define SERVER_IP "127.0.0.1"
#define SERVER_PORT 5670
void main()
{
    struct sockaddr_in client, server;
    int sd, nsd, clen = sizeof(client);
    char msg[512];
    bzero((char *)&server, sizeof(server));
```

```
    server.sin_family = AF_INET;
    server.sin_port = htons(SERVER_PORT);
    server.sin_addr.s_addr = inet_addr(SERVER_IP);
    sd = socket(AF_INET, SOCK_STREAM, 0);
    bind(sd, (struct sockaddr *)&server, sizeof(server));
    listen(sd, 5);
    while (1)
    {
        nsd = accept(sd, (struct sockaddr *)&client, &clen);
        do
        {
            memset(msg, 0x0, 512);
            recv(nsd, msg, 512, 0);
            printf("\nMessage received from Client: %s, Client IP: %s, Client Port: %u\n", msg,
inet_ntoa(client.sin_addr), ntohs(client.sin_port));
        } while (strcmp(msg, "stop"));
        close(nsd);
    }
}
```

Output

client.c:

```
Enter a Message: Hello world
Enter a Message: Enter a Message: stop
```

server.c:

```
Message received from Client: Hello, Client IP: 127.0.0.1, Client Port: 54466

Message received from Client: stop, Client IP: 127.0.0.1, Client Port: 54466
```

Assignment 5

Problem Statement

Write a program to develop an Iterative ECHO server using TCP socket.

Code

In client.c:

```
#include <stdio.h>
#include <string.h>
#include <sys/socket.h>
#include <sys/types.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <unistd.h>
#define CLIENT_IP "127.0.0.1"
#define CLIENT_PORT 5671
#define SERVER_IP "127.0.0.1"
#define SERVER_PORT 5670
void main()
{
    struct sockaddr_in client, server;
    int sd;
    char msg[512], msg1[512];
    bzero((char *)&server, sizeof(server));
    server.sin_family = AF_INET;
    server.sin_port = htons(SERVER_PORT);
    server.sin_addr.s_addr = inet_addr(SERVER_IP);
    bzero((char *)&client, sizeof(client));
    client.sin_family = AF_INET;
    client.sin_port = htons(CLIENT_PORT);
    client.sin_addr.s_addr = inet_addr(CLIENT_IP);
    sd = socket(AF_INET, SOCK_STREAM, 0);
    connect(sd, (struct sockaddr *)&server, sizeof(server));
```

```
    do
    {
        printf("Enter a Message: ");
        scanf("%s", msg);
        send(sd, msg, strlen(msg) + 1, 0);
        memset(msg1, 0x0, 512);
        recv(sd, msg1, 512, 0);
        printf("\nMessage received from Server: %s\n", msg1);
    } while (strcmp(msg, "stop"));
    close(sd);
}
```

In server.c:

```
#include <stdio.h>
#include <string.h>
#include <sys/socket.h>
#include <sys/types.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <unistd.h>
#define SERVER_IP "127.0.0.1"
#define SERVER_PORT 5670
void main() {
    struct sockaddr_in client, server;
    int sd, nsd, clen = sizeof(client);
    char msg[512];
    bzero((char *)&server, sizeof(server));
    server.sin_family = AF_INET;
    server.sin_port = htons(SERVER_PORT);
    server.sin_addr.s_addr = inet_addr(SERVER_IP);
    sd = socket(AF_INET, SOCK_STREAM, 0);
    bind(sd, (struct sockaddr *)&server, sizeof(server));
    listen(sd, 5);
    while (1) {
```

```
        nsd = accept(sd, (struct sockaddr *)&client, &clen);
        do {
            memset(msg, 0x0, 512);
            recv(nsd, msg, 512, 0);
            send(nsd, msg, strlen(msg) + 1, 0);
        } while (strcmp(msg, "stop"));
        close(nsd);
    }
}
```

Output

```
Enter a Message: Ping

Message received from Server: Ping
Enter a Message: stop

Message received from Server: stop
```

Assignment 6

Problem Statement

Write a program to develop a Iterative Day- Time server using TCP socket.

Code

In client.c:

```
#include <stdio.h>
#include <string.h>
#include <sys/socket.h>
#include <sys/types.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <unistd.h>
#define CLIENT_IP "127.0.0.1"
#define CLIENT_PORT 5671
#define SERVER_IP "127.0.0.1"
#define SERVER_PORT 5670
void main()
{
    struct sockaddr_in client, server;
    int sd;
    char msg[512], msg1[512];
    bzero((char *)&server, sizeof(server));
    server.sin_family = AF_INET;
    server.sin_port = htons(SERVER_PORT);
    server.sin_addr.s_addr = inet_addr(SERVER_IP);
    bzero((char *)&client, sizeof(client));
    client.sin_family = AF_INET;
    client.sin_port = htons(CLIENT_PORT);
    client.sin_addr.s_addr = inet_addr(CLIENT_IP);
    sd = socket(AF_INET, SOCK_STREAM, 0);
    connect(sd, (struct sockaddr *)&server, sizeof(server));
```

```
    do
    {
        printf("Enter a Message: ");
        scanf("%s", msg);
        send(sd, msg, strlen(msg) + 1, 0);
        memset(msg1, 0x0, 512);
        recv(sd, msg1, 512, 0);
        printf("\nMessage received from Server: %s\n", msg1);
    } while (strcmp(msg, "stop"));
    close(sd);
}
```

<u>In server.c:</u>

```
#include <stdio.h>
#include <string.h>
#include <sys/socket.h>
#include <sys/types.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <unistd.h>
#include <time.h>
#define SERVER_IP "127.0.0.1"
#define SERVER_PORT 5670
void main()
{
    char *ptr;
    time_t ti;
    struct sockaddr_in client, server;
    int sd, nsd, clen = sizeof(client);
    char msg[512], msg1[] = "Bad Request";
    bzero((char *)&server, sizeof(server));
    server.sin_family = AF_INET;
    server.sin_port = htons(SERVER_PORT);
    server.sin_addr.s_addr = inet_addr(SERVER_IP);
```

```
    sd = socket(AF_INET, SOCK_STREAM, 0);
    bind(sd, (struct sockaddr *)&server, sizeof(server));
    listen(sd, 5);
    while (1)
    {
        nsd = accept(sd, (struct sockaddr *)&client, &clen);
        do
        {
            memset(msg, 0x0, 512);
            recv(nsd, msg, 512, 0);
            if (strcmp(msg, "time") == 0)
            {
                ti = time(NULL);
                ptr = ctime(&ti);
                send(nsd, ptr, strlen(ptr) + 1, 0);
            }
            else
            {
                send(nsd, msg1, strlen(msg1) + 1, 0);
            }
        } while (strcmp(msg, "stop"));
        close(nsd);
    }
}
```

Output

```
Enter a Message: Ping

Message received from Server: Bad Request
Enter a Message: time

Message received from Server: Sat Apr  5 13:47:34 2025
```

Assignment 7

Problem Statement

Write a program to develop an Iterative ECHO server using UDP socket.

Code

In client.c:

```
#include <stdio.h>
#include <string.h>
#include <sys/socket.h>
#include <sys/types.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <unistd.h>
#define CLIENT_IP "127.0.0.1"
#define CLIENT_PORT 6541
#define SERVER_IP "127.0.0.1"
#define SERVER_PORT 6540
void main() {
    struct sockaddr_in client, server;
    int sd, slen = sizeof(server);
    char msg[512], str[512];
    bzero((char *)&server, sizeof(server));
    server.sin_family = AF_INET;
    server.sin_addr.s_addr = inet_addr(SERVER_IP);
    server.sin_port = htons(SERVER_PORT);
    bzero((char *)&client, sizeof(client));
    client.sin_family = AF_INET;
    client.sin_addr.s_addr = inet_addr(CLIENT_IP);
    client.sin_port = htons(CLIENT_PORT);

    sd = socket(AF_INET, SOCK_DGRAM, 0);
    bind(sd, (struct sockaddr *)&client, sizeof(client));
```

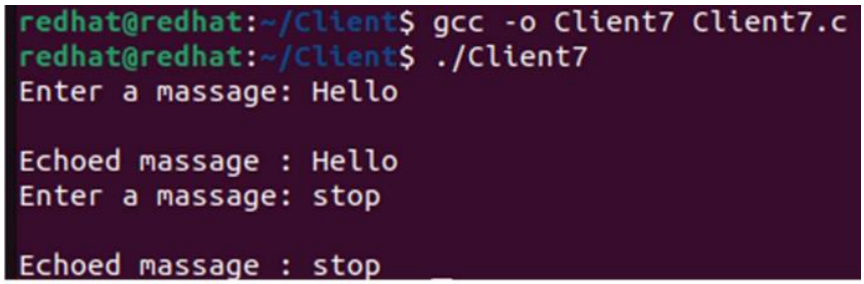
```
    do {
        printf("Enter a massage: ");
        scanf("%s", str);
        sendto(sd, str, strlen(str) + 1, 0, (struct sockaddr *)&server, sizeof(server));
        memset(msg, 0x0, 512);
        recvfrom(sd, msg, 512, 0, (struct sockaddr *)&server, &slen);
        printf("\nEchoed massage : %s\n", msg);

    } while (strcmp(str, "stop"));
    close(sd);
}
```

<u>In server.c:</u>

```
#include <stdio.h>
#include <string.h>
#include <sys/socket.h>
#include <sys/types.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <unistd.h>
#define SERVER_IP "127.0.0.1"
#define SERVER_PORT 6540
void main() {
    struct sockaddr_in client, server;
    int sd, clen = sizeof(client);
    char msg[512];
    bzero((char *)&server, sizeof(server));
    server.sin_family = AF_INET;
    server.sin_addr.s_addr = inet_addr(SERVER_IP);
    server.sin_port = htons(SERVER_PORT);
    sd = socket(AF_INET, SOCK_DGRAM, 0);
    bind(sd, (struct sockaddr *)&server, sizeof(server));
    while (1) {
        do {
```

```
            memset(msg, 0x0, 512);
            recvfrom(sd, msg, 512, 0, (struct sockaddr *)&client, &clen);
            sendto(sd, msg, strlen(msg) + 1, 0, (struct sockaddr *)&client, clen);
        } while (strcmp(msg, "stop"));
    }
}
```

Output



Assignment 8

Problem Statement

Write a program to develop an Iterative Day- Time server using UDP socket.

Code

In client.c:

```
#include <stdio.h>
#include <string.h>
#include <sys/socket.h>
#include <sys/types.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <unistd.h>
#define CLIENT_IP "127.0.0.1"
#define CLIENT_PORT 6541
#define SERVER_IP "127.0.0.1"
#define SERVER_PORT 6540
void main() {
```

```
    struct sockaddr_in client, server;
    int sd, slen = sizeof(server);
    char msg[512], str[512];
    bzero((char *)&server, sizeof(server));
    server.sin_family = AF_INET;
    server.sin_addr.s_addr = inet_addr(SERVER_IP);
    server.sin_port = htons(SERVER_PORT);
    bzero((char *)&client, sizeof(client));
    client.sin_family = AF_INET;
    client.sin_addr.s_addr = inet_addr(CLIENT_IP);
    client.sin_port = htons(CLIENT_PORT);

    sd = socket(AF_INET, SOCK_DGRAM, 0);
    bind(sd, (struct sockaddr *)&client, sizeof(client));
    do {
        printf("Enter a massage: ");
        scanf("%s", str);
        sendto(sd, str, strlen(str) + 1, 0, (struct sockaddr *)&server, sizeof(server));
        memset(msg, 0x0, 512);
        recvfrom(sd, msg, 512, 0, (struct sockaddr *)&server, &slen);
        printf("\nEchoed massage : %s\n", msg);

    } while (strcmp(str, "stop"));
    close(sd);
}
```

<u>In server.c:</u>

```
#include <stdio.h>
#include <string.h>
#include <sys/socket.h>
#include <sys/types.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <unistd.h>
```

```
#include <time.h>
#define SERVER_IP "127.0.0.1"
#define SERVER_PORT 6540
void main() {
    struct sockaddr_in client, server;
    int sd, clen = sizeof(client);
    char msg[512], str[512] = "Invalid request";
    char *ptr;
    time_t ti;
    bzero((char *)&server, sizeof(server));
    server.sin_family = AF_INET;
    server.sin_addr.s_addr = inet_addr(SERVER_IP);
    server.sin_port = htons(SERVER_PORT);
    sd = socket(AF_INET, SOCK_DGRAM, 0);
    bind(sd, (struct sockaddr *)&server, sizeof(server));
    while (1) {
        do {
            memset(msg, 0x0, 512);
            recvfrom(sd, msg, 512, 0, (struct sockaddr *)&client, &clen);
            if (!strcmp(msg, "time")) {
                ti = time(NULL);
                ptr = ctime(&ti);
                sendto(sd, ptr, strlen(ptr) + 1, 0, (struct sockaddr *)&client, clen);
            } else
                sendto(sd, str, strlen(str) + 1, 0, (struct sockaddr *)&client, clen);
        } while (strcmp(msg, "stop"));
    }
}
```

Output

```
redhat@redhat:~/Client$ gcc -o Client8 Client8.c
redhat@redhat:~/Client$ ./Client8
Enter a massage: time

Echoed massage : Sat Apr 19 13:55:38 2025

Enter a massage: stop

Echoed massage : Invalid request
```

Assignment 9

Problem Statement

Write a program to develop a Concurrent ECHO server using TCP socket. Use at least two client programs to implement it.

Code

In client_1.c:

```
#include <stdio.h>
#include <string.h>
#include <sys/socket.h>
#include <sys/types.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <unistd.h>
#define CLIENT_IP "127.0.0.2"
#define CLIENT_PORT 5981
#define SERVER_IP "127.0.0.1"
#define SERVER_PORT 5980
void main() {
    struct sockaddr_in client, server;
    int sd;
    char msg[512], msg1[512];
    bzero((char *)&server, sizeof(server));
    server.sin_family = AF_INET;
    server.sin_port = htons(SERVER_PORT);
    server.sin_addr.s_addr = inet_addr(SERVER_IP);
    bzero((char *)&client, sizeof(client));
    client.sin_family = AF_INET;
    client.sin_port = htons(CLIENT_PORT);
    client.sin_addr.s_addr = inet_addr(CLIENT_IP);
    sd = socket(AF_INET, SOCK_STREAM, 0);
    connect(sd, (struct sockaddr *)&server, sizeof(server));
```

```
    do {
        printf("Enter a Message: ");
        scanf("%s", msg);
        send(sd, msg, strlen(msg) + 1, 0);
        memset(msg1, 0x0, 512);
        recv(sd, msg1, 512, 0);
        printf("\nMessage received from Server: %s\n", msg1);
    } while (strcmp(msg, "stop"));
    close(sd);
}
```

In client_2.c:

```
#include <stdio.h>
#include <string.h>
#include <sys/socket.h>
#include <sys/types.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <unistd.h>
#define CLIENT_IP "127.0.0.3"
#define CLIENT_PORT 5982
#define SERVER_IP "127.0.0.1"
#define SERVER_PORT 5980
void main() {
    struct sockaddr_in client, server;
    int sd;
    char msg[512], msg1[512];
    bzero((char *)&server, sizeof(server));
    server.sin_family = AF_INET;
    server.sin_port = htons(SERVER_PORT);
    server.sin_addr.s_addr = inet_addr(SERVER_IP);
    bzero((char *)&client, sizeof(client));
    client.sin_family = AF_INET;
    client.sin_port = htons(CLIENT_PORT);
```

```
    client.sin_addr.s_addr = inet_addr(CLIENT_IP);
    sd = socket(AF_INET, SOCK_STREAM, 0);
    connect(sd, (struct sockaddr *)&server, sizeof(server));
    do{
        printf("Enter a Message: ");
        scanf("%s", msg);
        send(sd, msg, strlen(msg) + 1, 0);
        memset(msg1, 0x0, 512);
        recv(sd, msg1, 512, 0);
        printf("\nMessage received from Server: %s\n", msg1);
    } while (strcmp(msg, "stop"));
    close(sd);
}
```

In server.c:

```
#include <stdio.h>
#include <string.h>
#include <sys/socket.h>
#include <sys/types.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <unistd.h>
#include <stdlib.h>
#define SERVER_IP "127.0.0.1"
#define SERVER_PORT 5980
void main() {
    struct sockaddr_in client, server;
    int pid, sd, nsd, clen = sizeof(client);
    char msg[512];
    bzero((char *)&server, sizeof(server));
    server.sin_family = AF_INET;
    server.sin_port = htons(SERVER_PORT);
```

```
    server.sin_addr.s_addr = inet_addr(SERVER_IP);
    sd = socket(AF_INET, SOCK_STREAM, 0);
    bind(sd, (struct sockaddr *)&server, sizeof(server));
    listen(sd, 5);
    while (1) {
        nsd = accept(sd, (struct sockaddr *)&client, &clen);
        pid = fork();
        if (pid < 0) {
            printf("Error in fork()");
            exit(0);
        } else if (pid > 0) {
            close(nsd);
        } else {
            do {
                memset(msg, 0x0, 512);
                recv(nsd, msg, 512, 0);
                send(nsd, msg, strlen(msg) + 1, 0);
            } while (strcmp(msg, "stop"));
        }
    }
}
```

Output

client_1.c:

```
redhat@redhat:~/Client$ gcc -o Client9a Client9a.c
redhat@redhat:~/Client$ ./Client9a
Enter a Message: Hello

Message received from Server: Hello
Enter a Message: stop

Message received from Server: stop
```

client_2.c:

```
redhat@redhat:~/Client$ gcc -o Client9b Client9b.c
redhat@redhat:~/Client$ ./Client9b
Enter a Message: Hi

Message received from Server: Hi
Enter a Message: stop

Message received from Server: stop
```

Assignment 10

Problem Statement

Write a program to implement Sliding Window protocol.

Code

In client.c:

```
#include <sys/types.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <unistd.h>
#define CLIENT_IP "127.0.0.1"
#define CLIENT_PORT 6542
#define SERVER_IP "127.0.0.1"
#define SERVER_PORT 6555
void main() {
    struct sockaddr_in client, server;
    int sd, n, i, j, count = 0;
    char msg[512], msg1[512], ack[512];
    bzero((char *)&server, sizeof(server));
    server.sin_family = AF_INET;
    server.sin_addr.s_addr = inet_addr(SERVER_IP);
    server.sin_port = htons(SERVER_PORT);
    bzero((char *)&client, sizeof(client));
    client.sin_family = AF_INET;
    client.sin_addr.s_addr = inet_addr(CLIENT_IP);
    client.sin_port = htons(CLIENT_PORT);
    sd = socket(AF_INET, SOCK_STREAM, 0);
    connect(sd, (struct sockaddr *)&server, sizeof(server));
    do {
        printf("Enter a Message: ");
        scanf("%s", msg);
        printf("Enter Window Size: ");
```

```
        scanf("%d", &n);
        j = 0;
        for (i = 0; i < strlen(msg); i++) {
            if (j < n) {
                msg1[j++] = msg[i];
            }
            if (j == n || i == strlen(msg) - 1){
                msg1[j] = '\0';
                send(sd, msg1, strlen(msg1) + 1, 0);
                memset(ack, 0x0, 512);
                recv(sd, ack, 512, 0);
                printf("%s%d\n", ack, count++);
                j = 0;
            }
        }
    } while (strcmp(msg, "stop"));
    close(sd);
}
```

In server.c:

```
#include <stdio.h>
#include <string.h>
#include<sys/socket.h>
#include <sys/types.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <unistd.h>
#define SERVER_IP "127.0.0.1"
#define SERVER_PORT 6555
void main() {
    struct sockaddr_in client, server;
    int sd, nsd, clen = sizeof(client);
    char msg[512], ack[] = "Acknowledgement received";
    bzero((char *)&server, sizeof(server));
```

```
    server.sin_family = AF_INET;
    server.sin_addr.s_addr = inet_addr(SERVER_IP);
    server.sin_port = htons(SERVER_PORT);
    sd = socket(AF_INET, SOCK_STREAM, 0);
    bind(sd, (struct sockaddr *)&server, sizeof(server));
    listen(sd, 5);
    while (1) {
        nsd = accept(sd, (struct sockaddr *)&client, &clen);
        do {
            memset(msg, 0x0, 512);
            recv(nsd, msg, 512, 0);
            printf("\nData Receievd: %s\n", msg);
            send(nsd, ack, strlen(ack) + 1, 0);
        } while (strcmp(msg, "stop"));
        close(sd);
    }
}
```

Output

client.c:

```
redhat@redhat:~/Client$ gcc -o Client10 Client10.c
redhat@redhat:~/Client$ ./Client10
Enter a Message: MCKVIE
Enter Window Size: 2
Acknowledgement received0
Acknowledgement received1
Acknowledgement received2
Enter a Message: ASDFGHJKL
Enter Window Size: 3
Acknowledgement received3
Acknowledgement received4
Acknowledgement received5
Enter a Message: ^C
```

server.c:

```
redhat@redhat:~/Server$ gcc -o Server10 Server10.c
redhat@redhat:~/Server$ ./Server10

Data Receievd: MC

Data Receievd: KV

Data Receievd: IE

Data Receievd: ASD

Data Receievd: FGH

Data Receievd: JKL

Data Receievd:

Data Receievd:
```